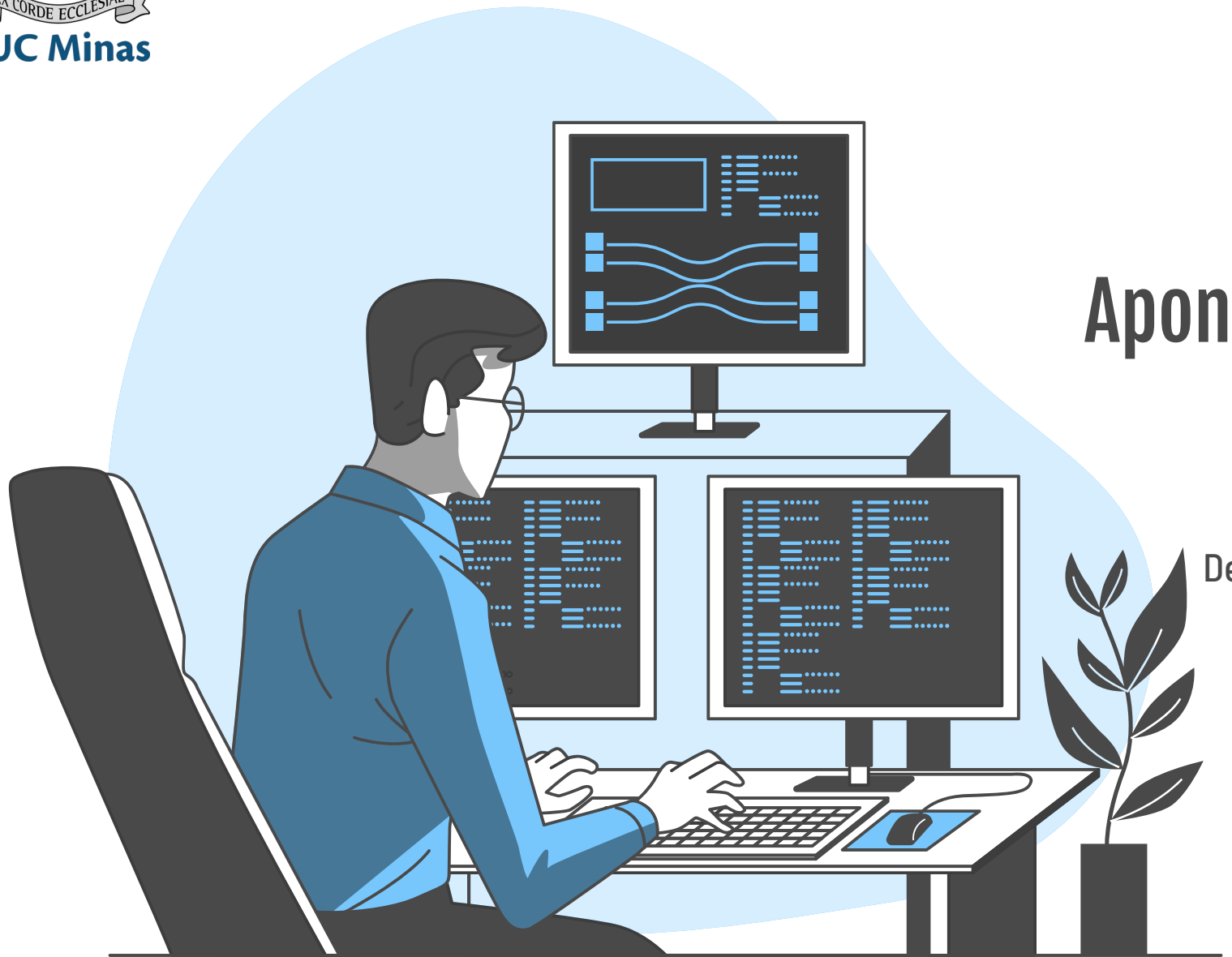
PUC Minas



Aula 07 · Deploy

Apontando um projeto para o outro: Vercel ⇄ Render

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender as **duas setas** que ligam front e back em produção

2

Configurar `VITE_API_URL` no Vercel e fazer **Redeploy**

3

Configurar `CORS_ORIGINS` no Render

4

Garantir o acesso do Render ao **MongoDB Atlas** e ao **Supabase**

5

Validar o fluxo completo com o DevTools

Antes de começar

- Aula 05 (Vercel) e aula 06 (Render)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 A ordem que evita o “ovo e a galinha”

02 Seta 1: o front aponta para o back (Vercel)

03 Seta 2: o back aceita o front (Render)

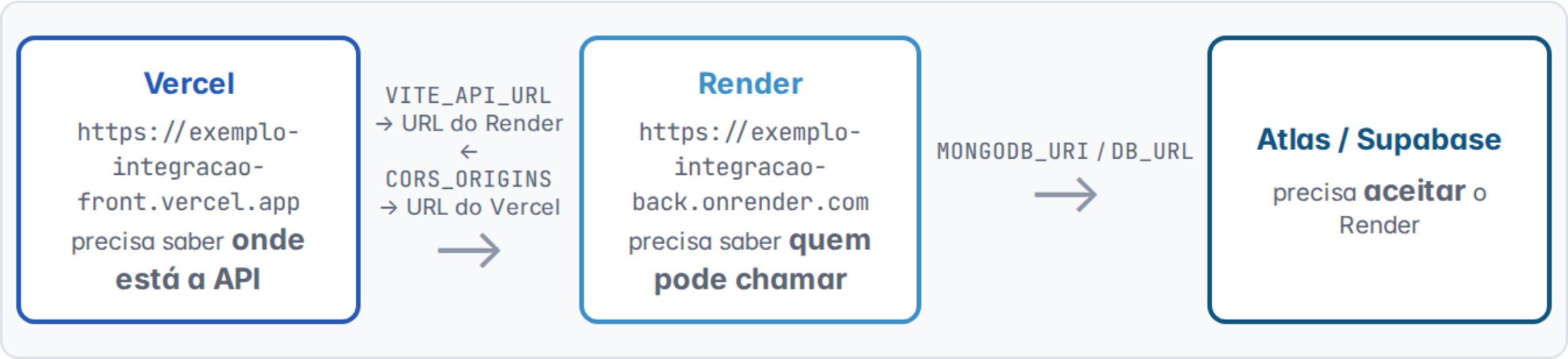
04 Seta 3: o banco aceita o Render

05 Teste final

06 Domínio próprio (opcional)

07 Aprofundando: um ambiente de homologação

Introdução



As três ligações de produção. Cada uma é uma variável de ambiente.

01

A ordem que evita o “ovo e a galinha”

A ordem que evita o “ovo e a galinha”

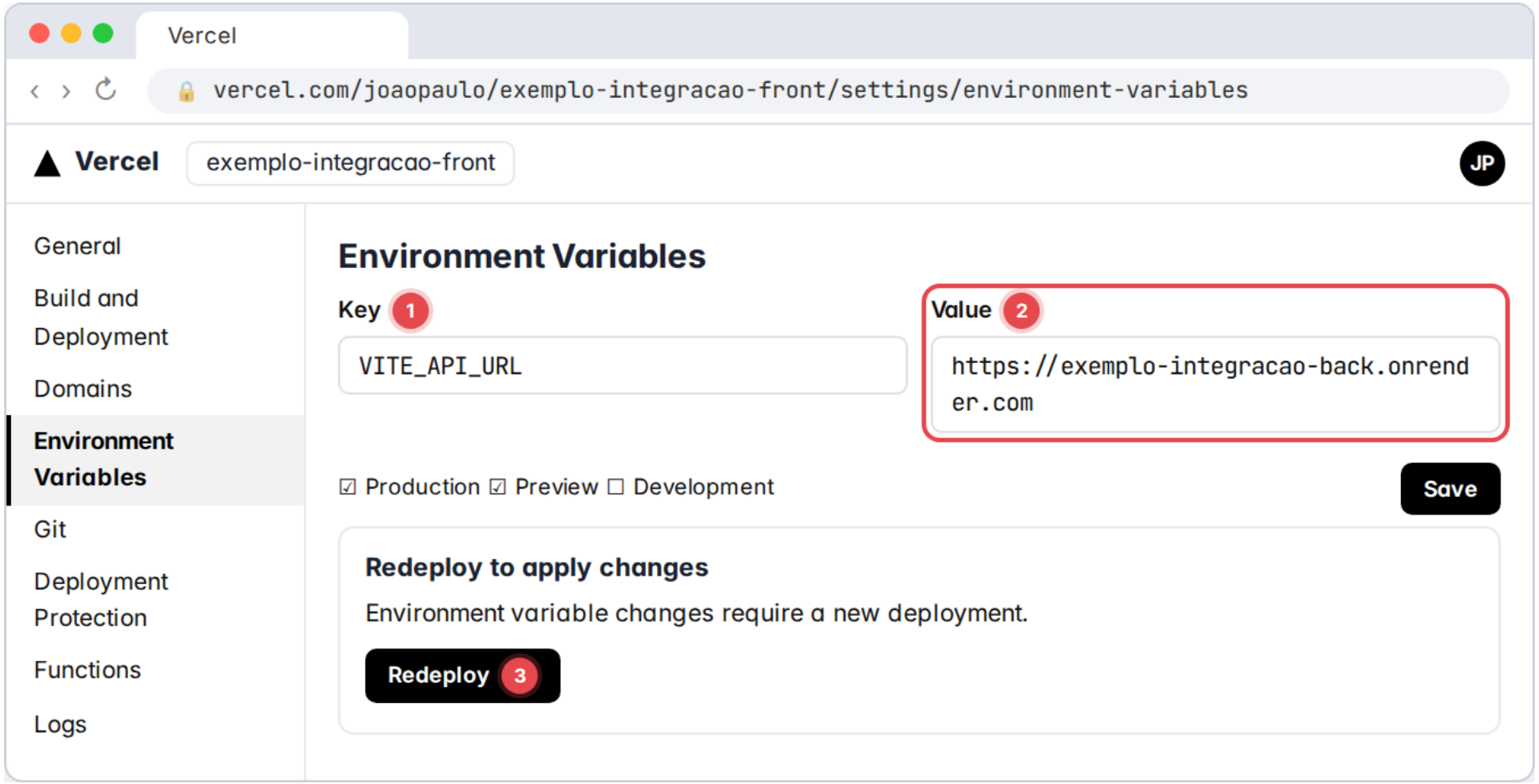
O front precisa da URL do back, e o back precisa da URL do front. Resolva assim:

1. Publique o **back** no Render com `CORS_ORIGINS=http://localhost:5173` (provisório). Anote a URL: `https://exemplo-integracao-back.onrender.com`.
2. Publique o **front** no Vercel com `VITE_API_URL` = URL do Render. Anote a URL: `https://exemplo-integracao-front.vercel.app`.
3. Volte ao Render e **acrescente** a URL do Vercel ao `CORS_ORIGINS`. Ao salvar, o Render faz um novo deploy sozinho.
4. Abra o site do Vercel e teste com o DevTools aberto.

02

Seta 1: o front aponta para o back (Vercel)

Seta 1: o front aponta para o back (Vercel)



No Vercel: a URL do Render, **https**, **sem barra no final**. Depois, **Redeploy**.

Seta 1: o front aponta para o back (Vercel)

O que o build do Vercel grava no JavaScript

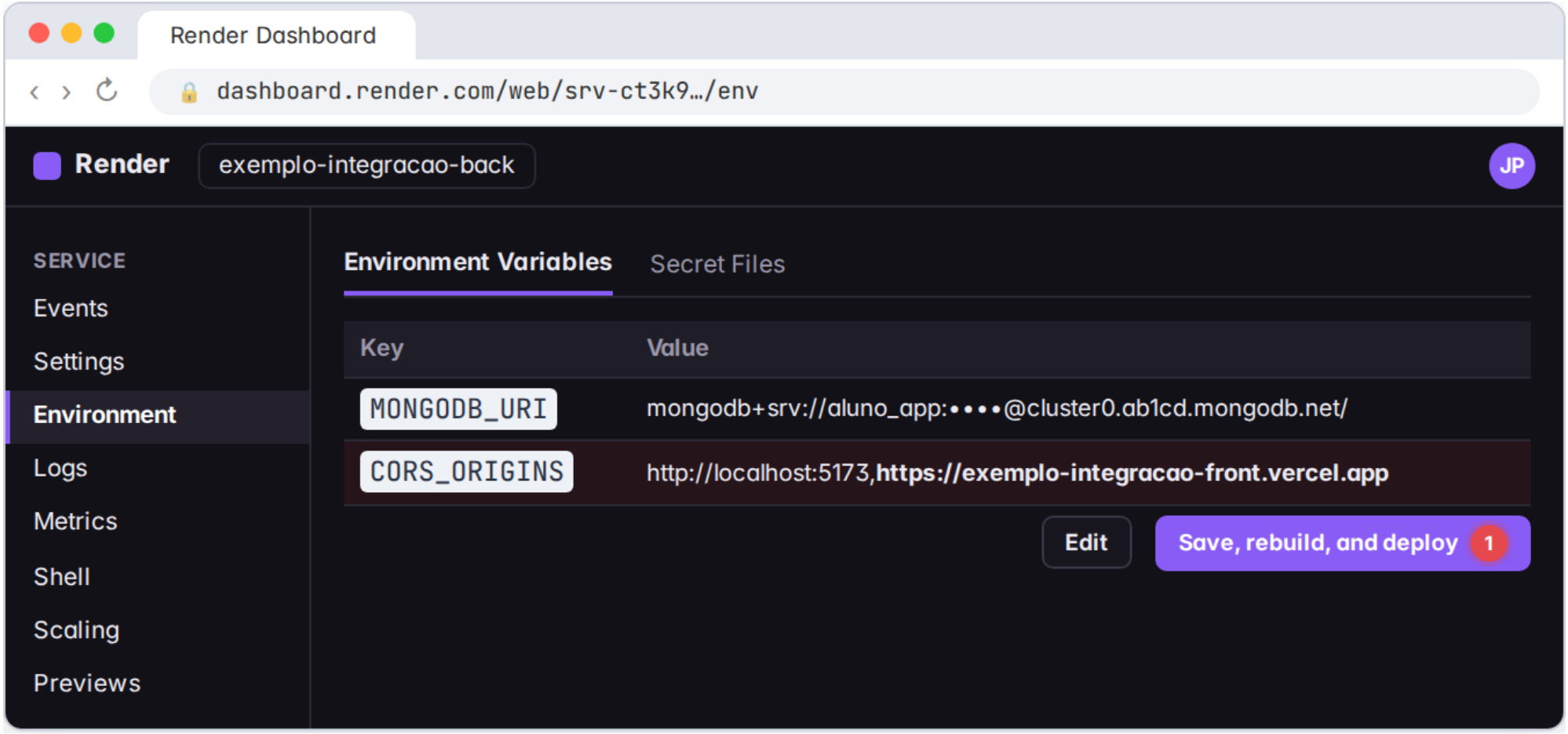
```
// config/apiConfig.js (FullStack, aula 04)
BASE_URL: (import.meta.env.VITE_API_URL || "http://localhost:8080").replace(/\/+$/, "")

// em produção vira, dentro do JS publicado:
BASE_URL: "https://exemplo-integracao-back.onrender.com"
```

03

Seta 2: o back aceita o front (Render)

Seta 2: o back aceita o front (Render)



No Render: a URL do Vercel entra na lista do `CORS_ORIGINS`, separada por vírgula.

Seta 2: o back aceita o front (Render)

Back: application.properties

```
app.cors.allowed-origins=${CORS_ORIGINS:http://localhost:5173}
# CorsProperties transforma a lista separada por vírgula em List<String>
# e o CorsConfig usa allowedOriginPatterns (aceita https://*.vercel.app)
```

Seta 2: o back aceita o front (Render)

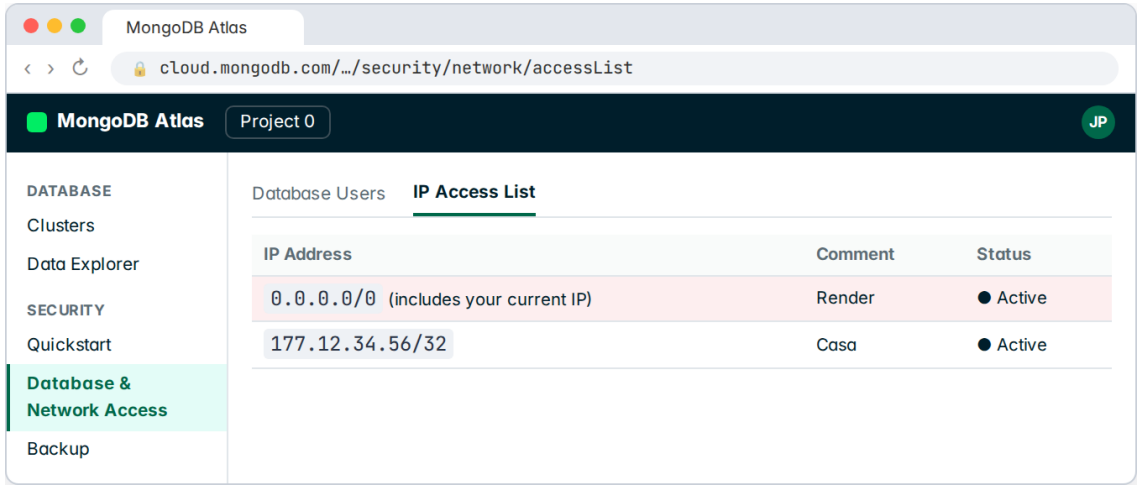
Atenção

As origens são comparadas **letra por letra**: `https://exemplo-integracao-front.vercel.app` funciona; `https://exemplo-integracao-front.vercel.app/` (com barra) e `http://...` (sem s) **não**. Para aceitar também os *previews* do Vercel, acrescente `https://*.vercel.app` (o `allowedOriginPatterns` aceita curingas).

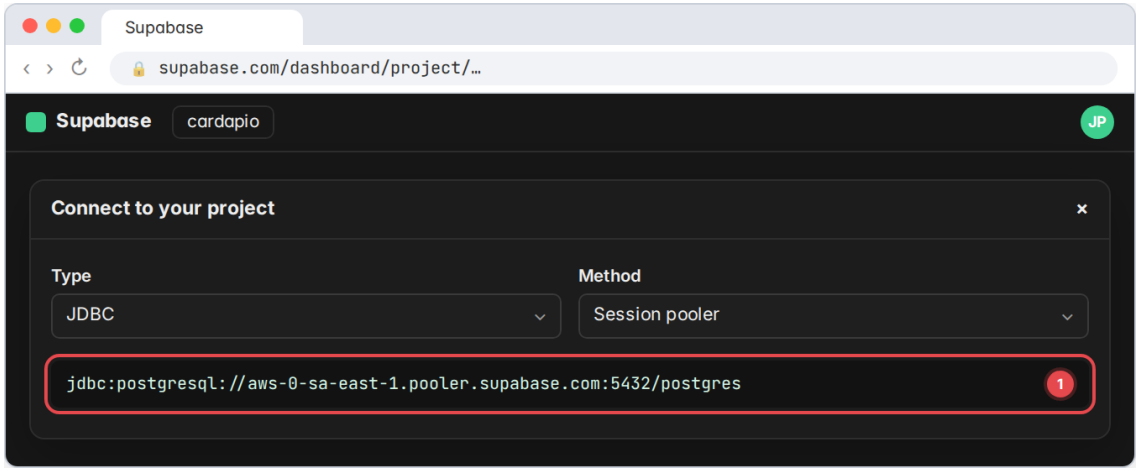
04

Seta 3: o banco aceita o Render

Seta 3: o banco aceita o Render



MongoDB Atlas: `0.0.0.0/0` no IP Access List, porque o Render Free não tem IP fixo.



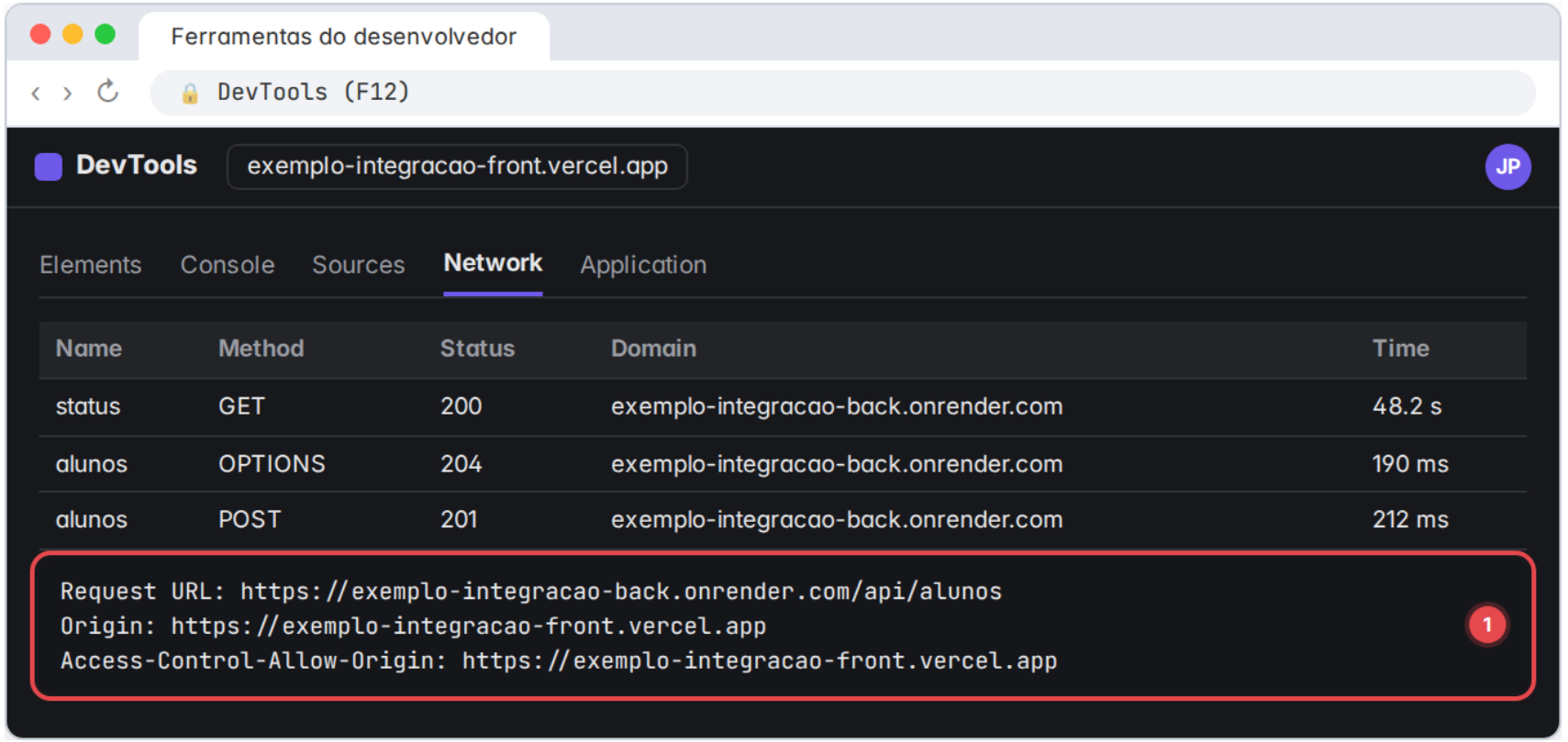
Supabase: use o *Session pooler* (IPv4). A *Direct connection* é IPv6 e falha no Render.

05

Teste final

Teste final

Abra o site do **Vercel**, pressione `F12` e vá à aba **Network**. Faça uma ação que chame a API (listar, cadastrar).



Tudo certo: o domínio chamado é o do Render, o `Origin` é o do Vercel e a resposta autoriza essa origem. Os 48 s da primeira chamada são o Render acordando.

Teste final

O que você vê	Diagnóstico
Domínio `localhost:8080` na aba Network	`VITE_API_URL` não chegou ao build: cadastre e faça **Redeploy**
Erro de CORS no Console	URL do Vercel ausente/errada no `CORS_ORIGINS` do Render
`GET status` pendente por ~50 s, depois `200`	Normal: *cold start* do Render Free
`503` com `mensagem` sobre banco	Back no ar, mas sem acesso ao Atlas/Supabase (seta 3)
`Mixed Content`	`VITE_API_URL` começa com `http://`

06

Domínio próprio (opcional)

Domínio próprio (opcional)

Com um domínio seu (ex.: `meuprojeto.dev`), crie dois registros **CNAME** no seu provedor de DNS, como no README do `PalpiteDTI`:

Tipo	Nome	Aponta para	Onde configurar
`CNAME`	`app`	`cname.vercel-dns.com`	Vercel → *Settings* → *Domains*
`CNAME`	`api`	`exemplo-integracao-back.onrender.com`	Render → *Settings* → *Custom Domains*

Domínio próprio (opcional)

Depois troque as variáveis: `VITE_API_URL=https://api.meuprojeto.dev` (Vercel, com Redeploy) e acrescente `https://app.meuprojeto.dev` ao `CORS_ORIGINS` (Render).

06

Domínio próprio (opcional)

Aula 07 · Deploy

07

Aprofundando: um ambiente de homologação

Dois ambientes, quatro serviços

Front produção

Vercel, branch `main`, `VITE_API_URL` → back de produção.

Front preview

Vercel, outras branches, `VITE_API_URL` (Preview) → back de staging.

Back produção

Render, branch `main`, banco oficial.

Back staging

Segundo serviço no Render, branch `develop`, banco de teste.

Aprofundando: um ambiente de homologação

Variável	Vercel Production	Vercel Preview
`VITE_API_URL`	`https://app-api.onrender.com`	`https://app-api-staging.onrender.com`
`VITE_SUPABASE_URL`	projeto oficial	projeto de teste

Aprofundando: um ambiente de homologação

CORS por ambiente

```
# Render: back de produção
CORS_ORIGINS=https://app.vercel.app

# Render: back de staging (aceita todos os previews do projeto)
CORS_ORIGINS=http://localhost:5173,https://app-*.vercel.app
```

Um banco para tudo x um banco por ambiente

Banco único

- Mais simples no plano gratuito
- Teste pode apagar dados reais
- Aceitável em trabalhos de aula

vs

Banco por ambiente

- Dados de teste isolados
- Dois clusters/projetos (Atlas e Supabase permitem no Free)
- Padrão em empresas

Aprofundando: um ambiente de homologação

1. Crie a branch `develop` e um segundo Web Service no Render apontando para ela.
2. Crie um banco de teste e coloque a connection string no serviço de staging.
3. No Vercel, defina `VITE_API_URL` diferente para *Production* e *Preview*.
4. Abra um PR de `develop` para `main`: o preview usa o back de staging.
5. Depois de revisar, mescle: a produção atualiza sozinha.

Aprofundando: um ambiente de homologação

Observação

Para trabalhos da disciplina, um único ambiente costuma bastar. Monte o de homologação quando o projeto tiver usuários reais ou quando várias pessoas estiverem mexendo ao mesmo tempo.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual seta liga o Render ao Vercel?

A VITE_API_URL

B CORS_ORIGINS com a URL do Vercel

C MONGODB_URI

D PORT

Resposta

Qual seta liga o Render ao Vercel?

A VITE_API_URL

✓ CORS_ORIGINS com a URL do Vercel

C MONGODB_URI

D PORT

Por quê: O back precisa saber quem pode chamá-lo.

Pergunta

Depois de colocar a URL do Render no Vercel, o que fazer?

A Nada

B Redeploy no Vercel

C Reiniciar o Atlas

D Apagar o cache do navegador

Resposta

Depois de colocar a URL do Render no Vercel, o que fazer?

A Nada

✓ Redeploy no Vercel

C Reiniciar o Atlas

D Apagar o cache do navegador

Por quê: No Render, salvar a variável já redeploya.

Pergunta

Por que o Atlas precisa de 0.0.0.0/0?

A Para o Compass

B O Render Free não tem IP fixo

C Para o Vercel

D Por causa do HTTPS

Resposta

Por que o Atlas precisa de 0.0.0.0/0?

A Para o Compass

✓ O Render Free não tem IP fixo

C Para o Vercel

D Por causa do HTTPS

Por quê: A proteção fica no usuário e senha do banco.

Pergunta

Na aba Network, as chamadas vão para localhost:8080 no site publicado. Causa:

A CORS

B VITE_API_URL ausente no build

C Banco offline

D Cold start

Quiz · questão 4 de 5

Resposta

Na aba Network, as chamadas vão para localhost:8080 no site publicado. Causa:

A CORS

✓ VITE_API_URL ausente no build

C Banco offline

D Cold start

Por quê: Cadastre e faça Redeploy.

Pergunta

Como aceitar também os previews do Vercel no CORS?

A allowedOrigins("*")

B Acrescentar https://*.vercel.app (allowedOriginPatterns)

C Desligar o CORS

D Usar http

Resposta

Como aceitar também os previews do Vercel no CORS?

A allowedOrigins("*")

✓ Acrescentar https://*.vercel.app (allowedOriginPatterns)

C Desligar o CORS

D Usar http

Por quê: Curingas exigem allowedOriginPatterns.

Fechamento

Resumo da aula

- ✓ Três ligações: Vercel → Render (`VITE_API_URL`), Render ← Vercel (`CORS_ORIGINS`), Render → banco.
- ✓ Ordem: back no Render → front no Vercel com a URL do back → URL do front no CORS do back.
- ✓ Variável do Vercel só vale após **Redeploy**; no Render, salvar já redeploya.
- ✓ Origens exatas: `https`, sem barra final; `https://*.vercel.app` cobre os previews.
- ✓ Atlas com `0.0.0.0/0`; Supabase pelo *Session pooler*. Valide tudo na aba Network.

Exercícios

1. Ligue o seu front do Vercel ao seu back do Render e grave um vídeo do CRUD funcionando em produção.

2. Remova a URL do Vercel do `CORS_ORIGINS`, capture o erro no Console e corrija.

3. Troque `VITE_API_URL` sem fazer Redeploy e explique o que acontece.

4. Abra um *preview deployment* e faça-o funcionar com o curinga `https://*.vercel.app`.

Para ir além

- README de `FullStack/PalpiteDTI` (seção Deploy) e de `FullStack/Exemplo_Integracao/Back`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 07?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web